

TAKT Industrial Risk Layer — финальная сборка и порядок запуска

Единая инструкция для подготовки демо к показу заказчику **Positive Technologies**.
Публичный адрес витрины: https://ralta.ru/TAKT_PT.

Демонстрируется ключевая метрика проекта — **время обработки данных SOC-оператором**:
полностью ручной режим (**≈ 29 мин 45 с**) против режима **TAKT** (**≈ 2 мин 55 с**),
ускорение **×10.2** на одном и том же наборе синтетических данных.

0. Что входит в финальную сборку

Компонент	Путь	Назначение
Фронтенд АРМ (React 18 + TS, строгий кибербез-UI)	frontend/	Операторская консоль: очередь инцидентов, карточка кейса, граф атаки, экран сравнения
Синтетический стенд (API)	stand/server.py	FastAPI, отдаёт контракт / api/v1/* + SSE + метрику
Генератор данных	stand/synthetic_data.py	Детерминированная многошаговая атака IEC-104 (6 кейсов, 486 событий)
Модель метрики	stand/benchmark.py	Параметрическая оценка времени оператора (ручной vs TAKT)
Docker Compose стенда	docker-compose.stand.yml	Одной командой поднимает связку frontend + API
Промпт для агента с доступом к ВМ	stand/DEPLOY_PROMPT_ralta.md	Пошаговый деплой на ralta.ru/TAKT_PT

- **Актуальная ветка сборки:** `feature/takt-pt-frontend-dark-ui` (финальный доработанный UI).
- **Проверено локально:** `tsc --noEmit` — без ошибок; `npm run build` — успешно (JS 657 КБ / gzip 213 КБ);
стенд `/health` — `ok`, 6 кейсов; связка фронтенд ↔ API работает (главная + `/compare`).

1. Локальный запуск — Вариант А: Docker (одна команда, рекомендуется)

Требования: Docker + плагин `docker compose`.

```
git clone --branch feature/takt-pt-frontend-dark-ui \
  https://github.com/Shushkary/takt-industrial-risk-layer.git
cd takt-industrial-risk-layer

docker compose -f docker-compose.stand.yml up --build
```

После старта откройте в браузере:

Что	Адрес
АРМ — очередь инцидентов (главная)	<code>http://localhost:3000</code>
Экран «Сравнение» (метрика оператора)	<code>http://localhost:3000/compare</code>
API стенда (Swagger)	<code>http://localhost:8090/docs</code>

Остановить: `Ctrl+C`, затем `docker compose -f docker-compose.stand.yml down`.

2. Локальный запуск — Вариант В: без Docker (для разработки/отладки)

Строгий порядок: **сначала API стенда, потом фронтенд** (фронтенд без API покажет пустую очередь).

Шаг 1. API стенда (терминал 1)

```
cd stand
pip install -r requirements.txt
uvicorn server:app --host 0.0.0.0 --port 8090
# проверка: curl -s http://127.0.0.1:8090/health → {"status":"ok","cases":6,...}
```

Шаг 2а. Фронтенд — режим разработки (терминал 2)

```
cd frontend
npm install
VITE_TAKT_API_BASE_URL=http://127.0.0.1:8090 npm run dev
# dev-сервер: http://localhost:3000
```

Шаг 26. Либо production-preview (ближе к бою)

```
cd frontend
npm install
VITE_TAKT_API_BASE_URL=http://127.0.0.1:8090 npm run build
npm run preview      # http://localhost:4173
```

`VITE_TAKT_API_BASE_URL` — адрес API стенда, который браузер вызывает напрямую (в `stand/server.py` CORS открыт на `*`). По умолчанию `http://127.0.0.1:8090`.

3. Публикация на ВМ — https://ralta.ru/TAKT_PT

Полный пошаговый пром프트 для агента с SSH-доступом к ВМ SpaceWeb лежит в `stand/DEPLOY_PROMPT_ralta.md` — скопируйте его целиком и передайте агенту с ключом.

Кратко порядок:

1. `ssh -i ~/.ssh/id_rsa_spaceweb torionadmin@89.111.142.231`, поставить Docker + compose-плагин.
2. `git clone --branch feature/takt-pt-frontend-dark-ui ...` в `/opt/takt/app`.
3. Собрать фронтенд под публичный API-путь:
`VITE_TAKT_API_BASE_URL="https://ralta.ru/TAKT_PT" docker compose -f docker-compose.stand.yml build`.
4. `docker compose -f docker-compose.stand.yml up -d` (frontend → :3000, API → :8090).
5. Прописать во внешнем nginx `ralta.ru` два location — `/TAKT_PT/` → :3000 и `/TAKT_PT/api/` → :8090 (с `proxy_buffering off` для SSE),
`nginx -t && systemctl reload nginx`.
6. Проверить публично `https://ralta.ru/TAKT_PT/` и `/compare`.

⚠ Сборочный агент (этот) не имеет SSH-ключа и 2FA-телефона ВМ, поэтому деплой на ВМ выполняет агент/человек с доступом. Весь код готов, собран и протестирован локально.

4. Порядок показа заказчику (сценарий демо, ~5 минут)

1. **Главная** — «Очередь инцидентов» (/).
Обратить внимание на операторскую консоль: живые UTC-часы, индикатор канала **LIVE · SSE**, смена оператора, KPI-полоса (всего/критические/в работе/новые), плотная **сортируемая** таблица инцидентов. Показать навигацию с клавиатуры: **j / k** — перемещение по строкам, **Enter** — открыть кейс. Клик по заголовку столбца — сортировка по риску/серьёзности/статусу.
2. **Карточка кейса** (Enter на верхнем критическом кейсе).
3-панельный **workbench**: XAI-резюме «почему это инцидент», граф атаки (цепочка IEC-104), таймлайн событий, baseline сущности (z-score).
3. **Экран «Сравнение»** (/compare) — кульминация.
Верхние карточки: **29:45 вручную** vs **2:55 TAKT**, экономия **26:50**, ускорение **×10.2**.

Нажать «**Начать разбор**» слева (ручной режим) — секундомер, надо найти 6 событий атаки среди ~480 шумовых. Затем справа режим ТАКТ — готовый коррелированный кейс, «**Подтвердить инцидент**». Живая иллюстрация метрики на одних и тех же данных.

5. Чек-лист готовности к показу

- [] `docker compose -f docker-compose.stand.yml up` поднимается без ошибок.
- [] `http://<host>:8090/health` → `{"status":"ok","cases":6}`.
- [] Главная / — таблица инцидентов заполнена, часы идут, индикатор **LIVE • SSE** зелёный.
- [] `j/k/Enter` и сортировка по столбцам работают.
- [] `/compare` — верхние карточки метрики заполнены (29:45 / 2:55 / ×10.2), оба секундомера стартуют.
- [] DevTools → Network: запросы `/api/v1/*` = 200, `stream/cases` держит SSE-соединение.
- [] (для BM) `https://ralta.ru/ТАКТ_PT/` и `/compare` открываются публично по HTTPS.

6. Эндпоинты API стенда

GET	<code>/api/v1/cases</code>	список кейсов
GET	<code>/api/v1/cases/{id}</code>	кейс
PATCH	<code>/api/v1/cases/{id}</code>	смена статуса
GET	<code>/api/v1/cases/{id}/attack-chain</code>	граф атаки
GET	<code>/api/v1/cases/{id}/events</code>	события кейса
GET	<code>/api/v1/events/search</code>	поиск (курсорная пагинация)
GET	<code>/api/v1/baseline/{type}/{id}</code>	baseline (z-score sparkline)
GET	<code>/api/v1/stream/cases</code>	SSE-поток новых кейсов
GET	<code>/api/v1/raw-events</code>	сырой поток для ручного режима
GET	<code>/api/v1/benchmark</code>	метрика: время оператора (JSON)
GET	<code>/api/v1/benchmark.md</code>	та же метрика в Markdown
GET	<code>/health</code>	проверка живости

Подробный разбор метрики — `docs/METRIC_OPERATOR_TIME.md`. Описание стенда — `stand/README.md`.